

```
"""
```

8-Puzzle Problem Implementation in Python

The 8-puzzle is a sliding puzzle that consists of a 3x3 grid with 8 numbered tiles and one blank space. The objective is to move the tiles around until they are in a specific goal configuration, typically:

```
|1|2|3|
|4|5|6|
|7|8|0|
```

```
"""
```

```
N = 3 # Size of the puzzle (3x3)
```

```
class PuzzleState:
```

```
    def __init__(self, board: list, x: int, y: int, depth: int):
```

```
        """
```

```
        Initialize the puzzle state.
```

```
        Args:
```

```
            board (list): The current configuration of the puzzle as a 2D list.
```

```
            x (int): The row index of the blank tile (0).
```

```
            y (int): The column index of the blank tile (0).
```

```
            depth (int): The depth of the current state in the search tree.
```

```
        """
```

```
        self.board = board
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.depth = depth
```

```
row_moves = [0, 0, -1, 1] # Left, Right, Up, Down
```

```
col_moves = [-1, 1, 0, 0] # Left, Right, Up, Down
```

```
def is_goal_state(board: list) -> bool:
```

```
    """
```

```
    Check if the current board configuration is the goal state.
```

```
    Args:
```

```
        board (list): The current configuration of the puzzle as a 2D list.
```

```
    Returns:
```

```
        bool: True if the board is the goal state, False otherwise.
```

```
    """
```

```
    # Define the goal state for the 8-puzzle
```

```
    goal = [[1, 2, 3], [4, 5, 6], [7, 8, 0]]
```

```
    # Check if the current board configuration matches the goal state
```

```
    return board == goal
```

```
def is_valid_move(x: int, y: int) -> bool:
```

```
    """
```

```
    Check if the move to the new position (x, y) is valid.
```

Args:

x (int): The row index of the new position.
y (int): The column index of the new position.

Returns:

bool: True if the move is valid, False otherwise.

"""

A move is valid if the new position is within the bounds of the board

return 0 <= x < N and 0 <= y < N

```
def print_board(board: list):
```

"""

Print the current board configuration.

Args:

board (list): The current configuration of the puzzle as a 2D list.

"""

Print the board in a readable format

for row in board:

print('|'.join(map(str, row)))

print("-----")

```
def solve_puzzle(start: list, x: int, y: int):
```

"""

Solve the 8-puzzle problem using Depth-First Search (DFS).

Args:

start (list): The initial configuration of the puzzle as a 2D list.

x (int): The row index of the blank tile (0).

y (int): The column index of the blank tile (0).

"""

stack = [] # To manage the states to explore

visited = set() # To keep track of visited states

Start with the initial state

stack.append(PuzzleState(start, x, y, 0))

Mark the initial state as visited (convert to tuple for immutability)

visited.add(tuple(map(tuple, start)))

DFS loop

while stack:

current_state = stack.pop() # Get the current state of the puzzle

Print the current state and its depth in the search tree

print(f"Depth: {current_state.depth}")

Print the current board configuration

print_board(current_state.board)

Check if we have reached the goal state

if is_goal_state(current_state.board):

If the current board configuration is the goal state,

print the solution found message and return

```

        print("Solution found at depth:", current_state.depth)
        return

# Generate the next states by moving the blank tile in all possible directions
for i in range(4):
    # Calculate the new row index for the blank tile
    new_x = current_state.x + row_moves[i]
    # Calculate the new column index for the blank tile
    new_y = current_state.y + col_moves[i]

    # Check if the move to the new position is valid
    if is_valid_move(new_x, new_y):
        # Create a new board configuration by swapping the blank tile with the
        # adjacent tile
        new_board = [row[:] for row in current_state.board]
        # Swap the blank tile (0) with the adjacent tile in the new position
        new_board[current_state.x][current_state.y], new_board[new_x][new_y] = \
            new_board[new_x][new_y], new_board[current_state.x][current_state.y]

        # Convert the new board configuration to a tuple for immutability and
        # to check if it has been visited
        board_tuple = tuple(map(tuple, new_board))

        # If the new board configuration has not been visited, mark it as visited and
        # add the new state to the stack for further exploration
        if board_tuple not in visited:
            visited.add(board_tuple)
            stack.append(PuzzleState(new_board, new_x, new_y, current_state.depth +
1))

# If we exhaust the stack without finding the goal state, print that no solution exists.
print("No solution found.")

def main():
    """
    Main function to initialize the puzzle and start the solving process.
    """
    # Define the initial board configuration and the position of the blank tile (0)
    start_board = [[1, 2, 3], [4, 0, 5], [6, 7, 8]]
    x, y = 1, 1 # Position of the blank tile (0)

    print("Initial State:")
    print_board(start_board)

    solve_puzzle(start_board, x, y)

if __name__ == "__main__":
    main()

```

Output:

```
# Initial State:
# 1|2|3
# 4|0|5
# 6|7|8
# -----
# Depth: 0
# 1|2|3
# 4|0|5
# 6|7|8
# -----
# Depth: 1
# 1|2|3
# 4|7|5
# 6|0|8
# -----
# Depth: 2
# 1|2|3
# 4|7|5
# 6|8|0
# -----
# Depth: 3
# 1|2|3
# 4|7|0
# 6|8|5
# -----
#.....
# Depth: 34838
# 1|2|3
# 4|5|6
# 7|8|0
# -----
# Solution found at depth: 34838
```